Laboratory Report: Lab 6

Zofia Warzycha, 280971

May 2026

1 Task Description

The goal of this lab was to extend the Space Station Management project by adding exception handling and sorting. The main things to implement were:

- Data validation using `IllegalArgumentException` in constructors and setters.
- Sorting crew members using `Comparable` and `Comparator` interfaces.
- A custom `InvalidSupplyException` for non-consumable supplies.
- A custom `ModuleFullException` when a module runs out of space.
- A custom `InvalidAccessCardDateException` for invalid card dates.
- A tester class that demonstrates all of the above with proper `try-catch` blocks.

2 Source Code

2.1 Custom Exception Classes

The three custom exceptions all extend `RuntimeException`, which means they are unchecked. This is the simplest approach – each one just passes the message to the parent constructor.

Listing 1: Custom exception classes

```
public class InvalidSupplyException extends RuntimeException {
    public InvalidSupplyException(String message) {
        super(message);
    }
}

public class ModuleFullException extends RuntimeException {
    public ModuleFullException(String message) {
        super(message);
    }
}

public class InvalidAccessCardDateException extends RuntimeException {
    public InvalidAccessCardDateException(String message) {
        super(message);
    }
}
```

All three follow the same pattern. I kept them as simple wrappers because the important part is the message that gets passed in from wherever the exception is thrown.

2.2 Data Validation with IllegalArgumentException

I added validation to constructors and setters in several classes. The idea is that if someone tries to create an object with bad data (like a negative age), the constructor throws an `IllegalArgumentException` immediately instead of letting the bad value in silently.

Listing 2: Validation in CrewMember constructor and setter

```
public CrewMember(String name, int age, Role role,
                  String specialization, double missionHours) {
    if (age < 0) {
        throw new IllegalArgumentException(
            "Age cannot be negative. Got: " + age);
    }
    if (missionHours < 0) {
        throw new IllegalArgumentException(
            "Mission hours cannot be negative. Got: " + missionHours);
    }
    this.id = UUID.randomUUID();
    this.name = name;
    this.age = age;
    this.role = role;
    this.specialization = specialization;
    this.missionHours = missionHours;
}

public void setAge(int age) {
    if (age < 0) {
        throw new IllegalArgumentException(
            "Age cannot be negative. Got: " + age);
    }
    this.age = age;
}
```

The same validation was also added to `Officer` (years of experience cannot be negative), `Supply` (quantity and cost cannot be negative), `Module` (capacity must be greater than zero), and `SpaceStation` (map dimensions and crew size must be positive).

2.3 Sorting – Comparable and Comparator

To sort crew members by age ascending, I made `CrewMember` implement `Comparable<CrewMember>`. The `compareTo` method just delegates to `Integer.compare`, which is the cleanest way to do it.

Listing 3: Comparable implementation in CrewMember

```
public class CrewMember implements Comparable<CrewMember> {

    // ... fields and other methods ...

    @Override
    public int compareTo(CrewMember other) {
        return Integer.compare(this.age, other.age);
    }
}
```

For the second sorting criterion (role descending, then mission hours ascending), I created a separate `Comparator` class. I used `b.getRole().compareTo(a.getRole())` to reverse the role order, and then `Double.compare` for hours.

Listing 4: CrewMemberRoleHoursComparator class

```java
import java.util.Comparator;

public class CrewMemberRoleHoursComparator implements Comparator<CrewMember> {

    @Override
    public int compare(CrewMember a, CrewMember b) {
        int roleCmp = b.getRole().compareTo(a.getRole());
        if (roleCmp != 0) {
            return roleCmp;
        }
        return Double.compare(a.getMissionHours(), b.getMissionHours());
    }
}
```

The reason I swapped `a` and `b` in the role comparison is that `Comparator` normally returns a negative value when the first argument comes first. By flipping them I get descending order for the role without any extra negation.

2.4 Supply – isConsumable and useSupply

I added a boolean field `isConsumable` to the `Supply` class, and a `getTotalPrice()` method that multiplies the unit cost by the quantity. I used `BigDecimal` for the cost to avoid floating-point rounding issues, which is important when dealing with money.

Listing 5: Supply class with isConsumable and getTotalPrice

```java
import java.math.BigDecimal;

public class Supply {

    private String name;
    private int quantity;
    private BigDecimal cost;
    private boolean isConsumable;

    public Supply(String name, int quantity, BigDecimal cost,
                  boolean isConsumable) {
        if (quantity < 0) {
            throw new IllegalArgumentException(
                "Quantity cannot be negative. Got: " + quantity);
        }
        if (cost.compareTo(BigDecimal.ZERO) < 0) {
            throw new IllegalArgumentException(
                "Cost cannot be negative. Got: " + cost);
        }
        this.name = name;
        this.quantity = quantity;
        this.cost = cost;
        this.isConsumable = isConsumable;
    }

    public BigDecimal getTotalPrice() {
        return cost.multiply(BigDecimal.valueOf(quantity));
    }

    public String getName()       { return name; }
    public int getQuantity()      { return quantity; }
    public BigDecimal getCost()   { return cost; }
    public boolean isConsumable() { return isConsumable; }
```

```
34
35      @Override
36      public String toString() {
37          return "Supply{name='" + name + "', qty=" + quantity
38                  + ", unitCost=" + cost + ", totalPrice=" + getTotalPrice()
39                  + ", consumable=" + isConsumable + "}";
40      }
41  }
```

The `useSupply` method was added to `CrewMember`. It checks the `isConsumable` flag and throws `InvalidSupplyException` if the supply cannot be consumed.

Listing 6: useSupply method in CrewMember

```
1   public void useSupply(Supply supply) throws InvalidSupplyException {
2       if (!supply.isConsumable()) {
3           throw new InvalidSupplyException(
4               "Supply \"" + supply.getName()
5               + "\" is not consumable and cannot be used by "
6               + this.name + ".");
7       }
8       System.out.println(this.name + " used supply: " + supply.getName()
9               + " (quantity: " + supply.getQuantity() + ").");
10  }
```

2.5 Module – ModuleFullException

The `addCrewMember` method in `Module` previously just printed a message when the module was full. Now it throws `ModuleFullException` instead, so the caller is forced to handle the situation.

Listing 7: addCrewMember with ModuleFullException in Module

```
1   public void addCrewMember(CrewMember newMember) throws ModuleFullException {
2       for (int i = 0; i < crewMembers.length; i++) {
3           if (crewMembers[i] == null) {
4               crewMembers[i] = newMember;
5               System.out.println(newMember.getName() + " added to module.");
6               return;
7           }
8       }
9       throw new ModuleFullException(
10          "Cannot add " + newMember.getName()
11          + ". Module is full (capacity: " + maxCapacity + ").");
12  }
```

The constructor also validates the capacity – you cannot create a module with zero or negative capacity.

Listing 8: Module constructor with validation

```
1   public Module(int maxCapacity) {
2       if (maxCapacity <= 0) {
3           throw new IllegalArgumentException(
4               "Module capacity must be greater than zero. Got: "
5               + maxCapacity);
6       }
7       this.maxCapacity = maxCapacity;
8       this.crewMembers = new CrewMember[maxCapacity];
9   }
```

2.6 AccessCard – InvalidAccessCardDateException

I added a second constructor to `AccessCard` that accepts a custom issue date. If the date is in the past, it throws `InvalidAccessCardDateException`.

Listing 9: AccessCard with date validation

```
import java.time.LocalDate;

public class AccessCard {

    private String accessLevel;
    private String crewMemberName;
    private LocalDate issueDate;

    public AccessCard(String accessLevel, String crewMemberName) {
        this.accessLevel = accessLevel;
        this.crewMemberName = crewMemberName;
        this.issueDate = LocalDate.now();
    }

    public AccessCard(String accessLevel, String crewMemberName,
                      LocalDate issueDate) {
        if (issueDate.isBefore(LocalDate.now())) {
            throw new InvalidAccessCardDateException(
                "Access card issue date cannot be in the past. Got: "
                + issueDate);
        }
        this.accessLevel = accessLevel;
        this.crewMemberName = crewMemberName;
        this.issueDate = issueDate;
    }

    public LocalDate getIssueDate() { return issueDate; }

    @Override
    public String toString() {
        return "[Access Card of " + crewMemberName + ", level: "
                + accessLevel + ", issued: " + issueDate + "]";
    }
}
```

For `PremiumAccessCard`, I added a second constructor that accepts a custom expiration date. If the expiration date is earlier than the issue date, an exception is thrown.

Listing 10: PremiumAccessCard with expiration date validation

```
public class PremiumAccessCard extends AccessCard {

    private LocalDate expirationDate;
    private String specialPrivileges;

    public PremiumAccessCard(String accessLevel, String crewMemberName,
                             String specialPrivileges) {
        super(accessLevel, crewMemberName);
        this.specialPrivileges = specialPrivileges;
        this.expirationDate = LocalDate.now().plusYears(1);
    }

    public PremiumAccessCard(String accessLevel, String crewMemberName,
                             String specialPrivileges,
```

```
                                LocalDate expirationDate) {
        super(accessLevel, crewMemberName);
        if (expirationDate.isBefore(getIssueDate())) {
            throw new InvalidAccessCardDateException(
                "Expiration date (" + expirationDate
                + ") cannot be earlier than issue date ("
                + getIssueDate() + ").");
        }
        this.specialPrivileges = specialPrivileges;
        this.expirationDate = expirationDate;
    }

    @Override
    public String toString() {
        return super.toString() + " -> PREMIUM [Expires: "
                + expirationDate + ", privileges: "
                + specialPrivileges + "]";
    }
}
```

2.7 Tester Class

The tester creates a mixed list of `CrewMember` objects (including subclasses), sorts them in two ways, tests `useSupply`, calculates total supply expenses, and tests all three custom exceptions.

Listing 11: Key sections from List6_tester.java

```
import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.*;

public class List6_tester {

    public static void main(String[] args) {

        // ---------- Crew list ----------
        List<CrewMember> crew = new ArrayList<>();
        crew.add(new CrewMember("Sebastian Kucharski", 22,
                Role.COMMANDER, "Organizational management", 9965.0));
        crew.add(new Engineer("Zofia Warzycha", 21,
                "Ship maintenance", 1500.5, "axe"));
        crew.add(new Scientist("Igor Stelmach", 21,
                "Astrophysics", 747.0, "dark matter"));
        crew.add(new Pilot("Stanislaw Blaszczyk", 74,
                "Long-range navigation", 850.5, 1200));
        crew.add(new Medic("Hanna Szpak", 20,
                "Emergency medicine", 668.0, "surgery"));
        crew.add(new Engineer("Michal Michalowski", 40,
                "Electrical systems", 3100.0, "multimeter"));

        // Sort by age (Comparable)
        System.out.println("\n---Crew sorted by age (ascending):");
        Collections.sort(crew);
        for (CrewMember m : crew) {
            System.out.println(m.getName() + ", age: " + m.getAge()
                + ", role: " + m.getRole()
                + ", hours: " + m.getMissionHours());
        }

```

```
        // Sort by role desc, hours asc (Comparator)
        System.out.println("\n---Crew sorted by role (desc) and hours (asc):");
        crew.sort(new CrewMemberRoleHoursComparator());
        for (CrewMember m : crew) {
            System.out.println(m.getName() + ", role: " + m.getRole()
                + ", hours: " + m.getMissionHours());
        }

        // ---------- Supply usage ----------
        System.out.println("\n---Supply usage and total costs:");
        Supply oxygenTank = new Supply("Oxygen Tank", 10,
                new BigDecimal("49.99"), true);
        Supply medKit = new Supply("Medical Kit", 4,
                new BigDecimal("120.00"), true);
        Supply spacesuit = new Supply("Spacesuit", 2,
                new BigDecimal("8500.00"), false);

        List<Supply> supplies = new ArrayList<>();
        supplies.add(oxygenTank);
        supplies.add(medKit);
        supplies.add(spacesuit);

        try {
            crew.get(0).useSupply(oxygenTank);
        } catch (InvalidSupplyException e) {
            System.out.println("Supply error: " + e.getMessage());
        }
        // invalid -- spacesuit is not consumable
        try {
            crew.get(0).useSupply(spacesuit);
        } catch (InvalidSupplyException e) {
            System.out.println("Supply error: " + e.getMessage());
        }

        BigDecimal total = BigDecimal.ZERO;
        for (Supply s : supplies) {
            total = total.add(s.getTotalPrice());
            System.out.println(s);
        }
        System.out.println("Total supply expenses: " + total + " USD");

        // ---------- Module capacity ----------
        System.out.println("\n---Module capacity test:");
        Module engineeringBay = new Module(3);
        try {
            engineeringBay.addCrewMember(crew.get(0));
            engineeringBay.addCrewMember(crew.get(1));
            engineeringBay.addCrewMember(crew.get(2));
            engineeringBay.addCrewMember(crew.get(3)); // should throw
        } catch (ModuleFullException e) {
            System.out.println("Module error: " + e.getMessage());
        }

        // ---------- Access cards ----------
        System.out.println("\n---Access card validation test:");
        AccessCard basicCard = new AccessCard("level 2", "Zofia Warzycha");
        System.out.println(basicCard);

        try {
```

```
            AccessCard pastCard = new AccessCard("level 1", "Hanna Szpak",
                    LocalDate.now().minusDays(5));
        } catch (InvalidAccessCardDateException e) {
            System.out.println("Access card error: " + e.getMessage());
        }

        PremiumAccessCard vipCard = new PremiumAccessCard(
                "level 10", "Michal Michalowski", "AllInclusive service");
        System.out.println(vipCard);

        try {
            PremiumAccessCard badPremium = new PremiumAccessCard(
                    "level 5", "Sebastian Kucharski",
                    "Command deck access",
                    LocalDate.now().minusMonths(2));
        } catch (InvalidAccessCardDateException e) {
            System.out.println("Premium card error: " + e.getMessage());
        }

        // ---------- Constructor validation ----------
        System.out.println("\n---Constructor validation test:");
        try {
            CrewMember invalid = new CrewMember("Bad Guy", -5,
                    Role.SCIENTIST, "Astrophysics", 0);
        } catch (IllegalArgumentException e) {
            System.out.println("Constructor error: " + e.getMessage());
        }

        try {
            Supply badSupply = new Supply("Broken item", -1,
                    new BigDecimal("10.00"), true);
        } catch (IllegalArgumentException e) {
            System.out.println("Supply error: " + e.getMessage());
        }

        try {
            Module tinyModule = new Module(0);
        } catch (IllegalArgumentException e) {
            System.out.println("Module error: " + e.getMessage());
        }
    }
}
```

3 Console Output

4 Conclusions

- I learned how to create custom exception classes by extending `RuntimeException`. It is really simple – you just pass the message to the parent constructor – but it makes the code much cleaner because the exception name itself already tells you what went wrong.
- I learned the difference between `Comparable` and `Comparator`. `Comparable` is implemented directly in the class and defines its natural ordering (in this case age ascending). `Comparator` is a separate class you pass to a sort method when you need a different or more complex ordering.
- I noticed that flipping the arguments in the role comparison (`b.compareTo(a)` instead of `a.compareTo(b)`) is a clean trick to get descending order without doing any extra work.

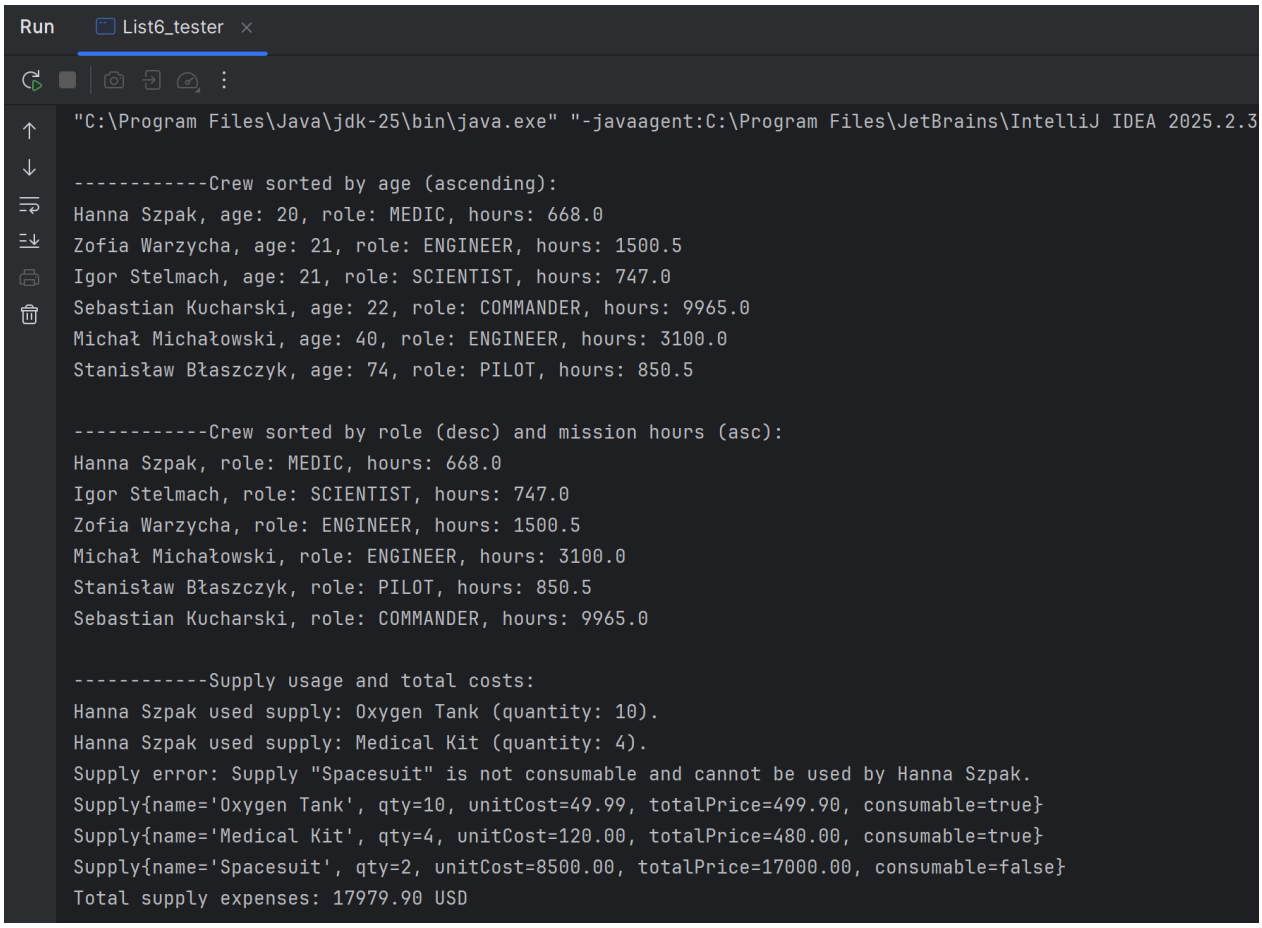

Figure 1: Console output – crew sorting and supply usage

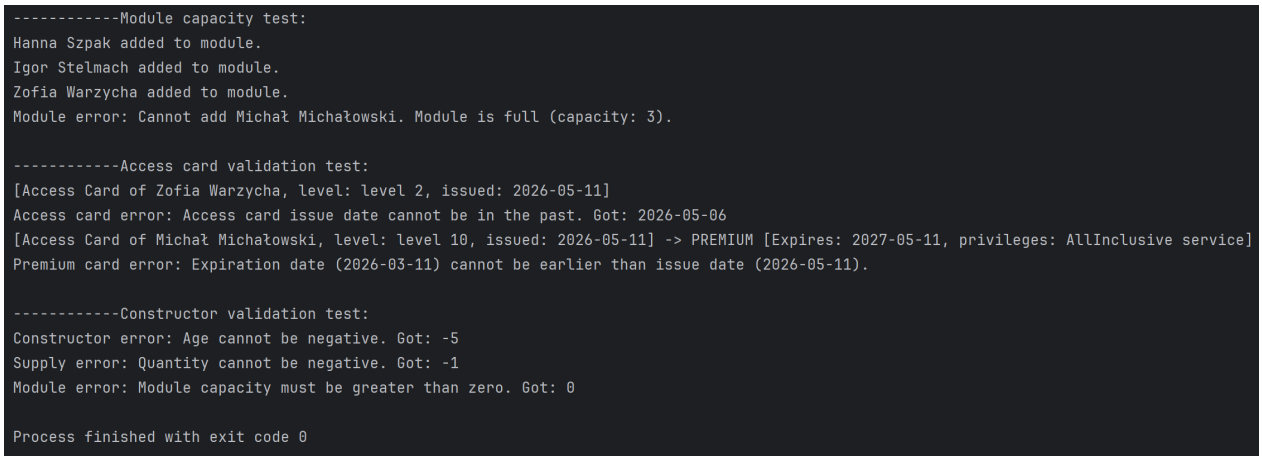

Figure 2: Console output – module, access card, and validation tests

- Using `BigDecimal` for the supply cost instead of `double` is the right choice whenever money is involved, because `double` can have small rounding errors that add up when multiplying.
- I found out that throwing an exception from a constructor is the safest way to prevent objects with invalid data from being created at all. The object never gets built, so there is no way for bad data to

leak into the rest of the program.

- Testing with `try-catch` blocks in the tester class showed me that the exceptions work correctly – the program does not crash, it catches the exception, prints the message, and continues with the rest of the tests.